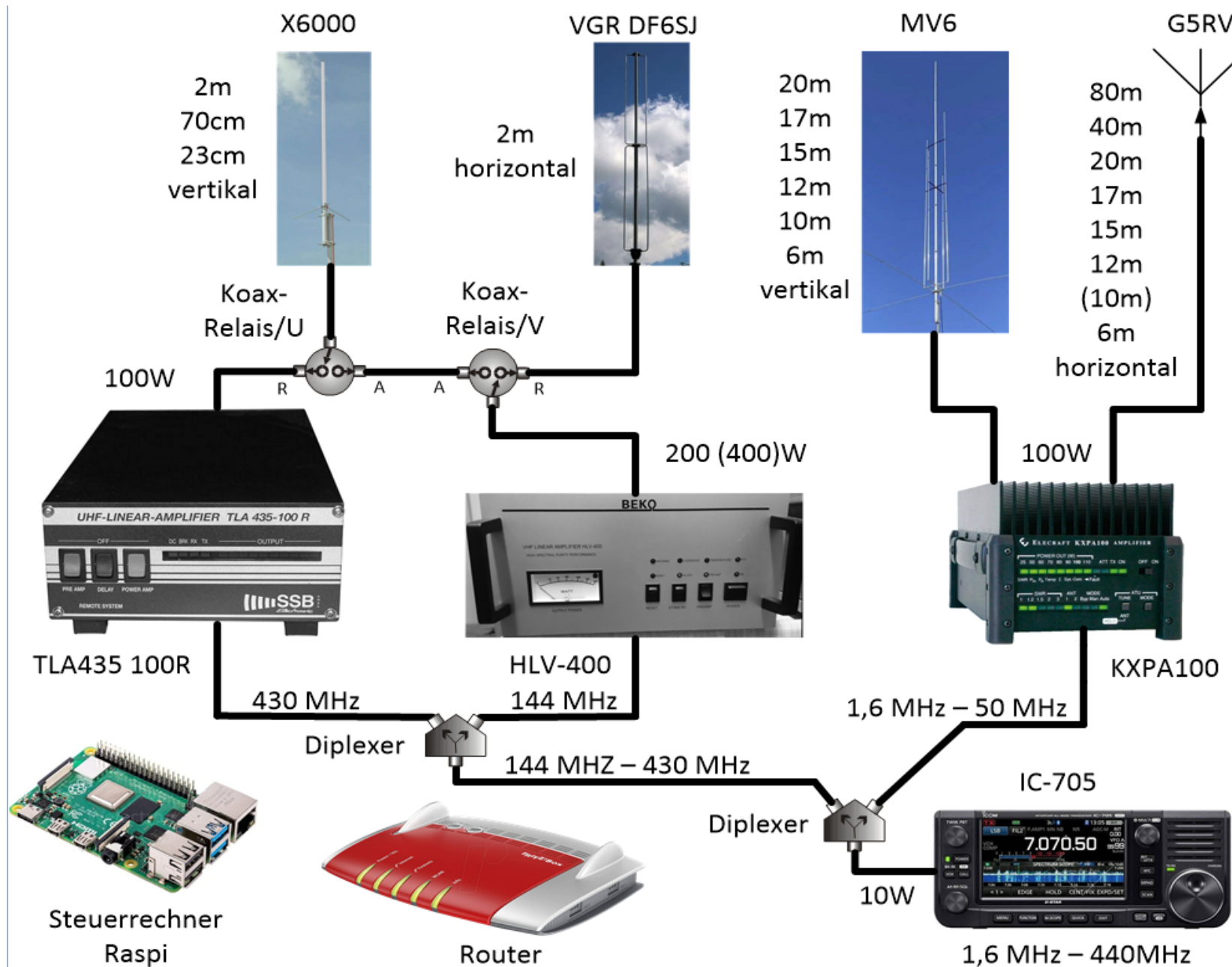


Beispiele CAT – Steuerung



- 1. ICOM-705 als Remote-Station
 - Steuerung über RaspberryPi
- 2. YAESU FT-710 mit externem Tuner
 - 2.1 Steuerung über PC
 - 2.2 Steuerung über ESP32
- Verweise

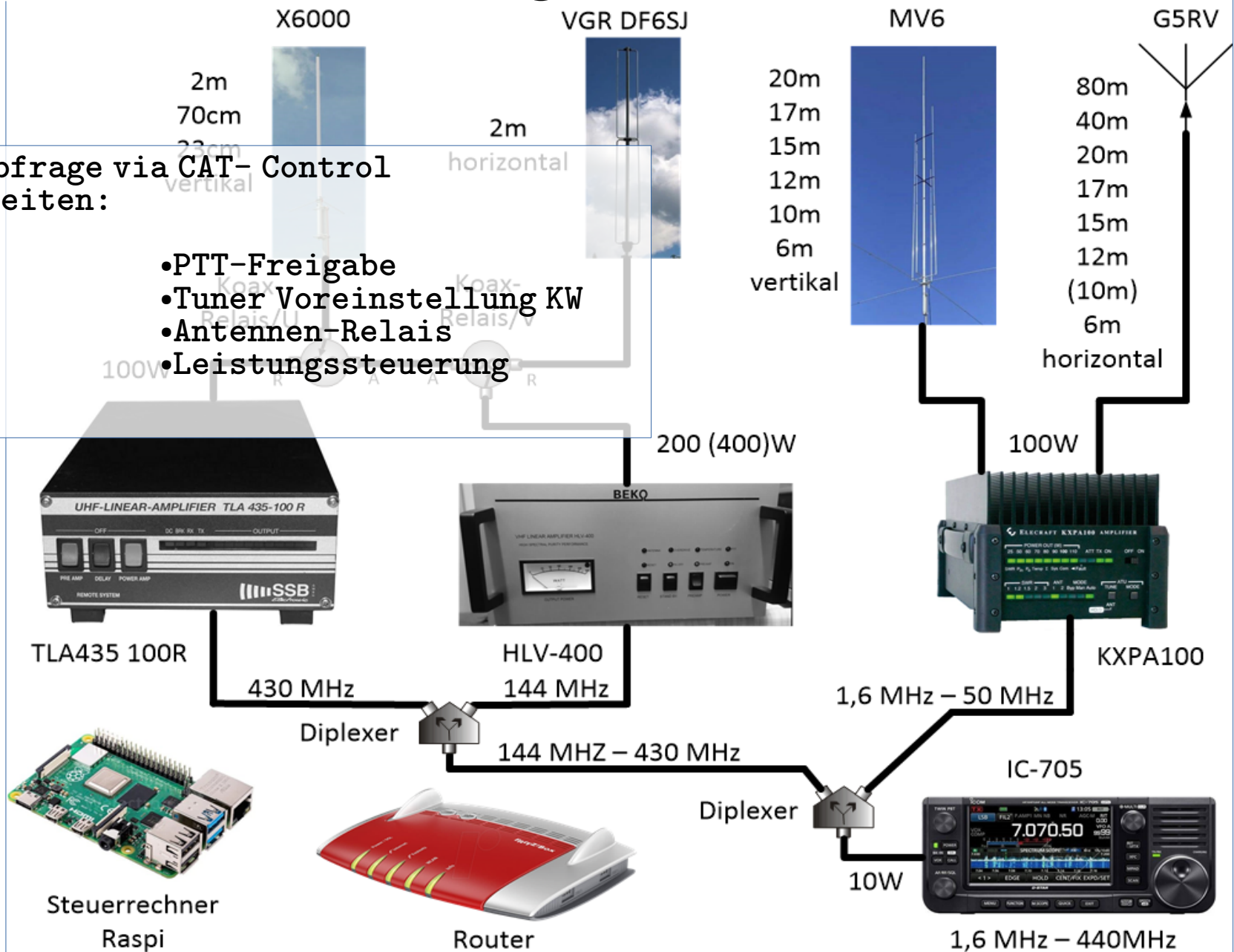
1. Remote- Station mit IC-705



Aufgabe der Steuerung

Frequenzabfrage via CAT- Control
Daraus ableiten:

- PTT-Freigabe
- Tuner Voreinstellung KW
- Antennen-Relais
- Leistungssteuerung



WEB-Interface „Dashboard“

*** Geräte Status ***

Frequenz : 14.015 MHz
Aktive PA : keine
Antenne : KW: G5RV/MV6

Info:
KXPA: SWR ---
ICOM: Kommunikation OK

*** Optionen ***

Antennen-Polarisation (UHF/VHF)

☒ vertikal ☐ horizontal

PA freigeben

☐ ja ☒ nein

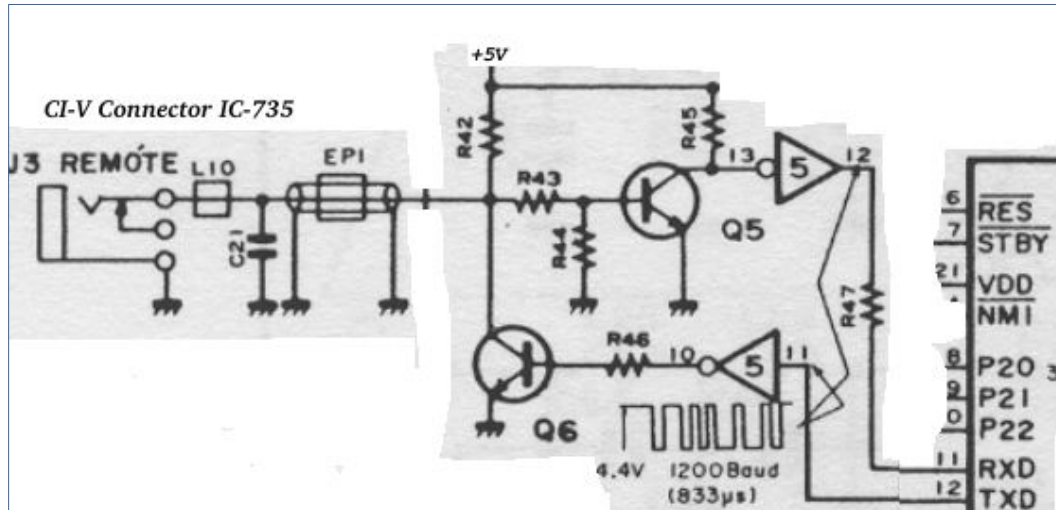
-Bei 2m vertical wird die Leistung
des Ic-705 auf 50% reduziert
-Auf 40m ist IMMER ANT1 (G5RV) gewählt.

Time [UTC]: 12.09.26 11:04:15
System Reboot @ 04:00 h

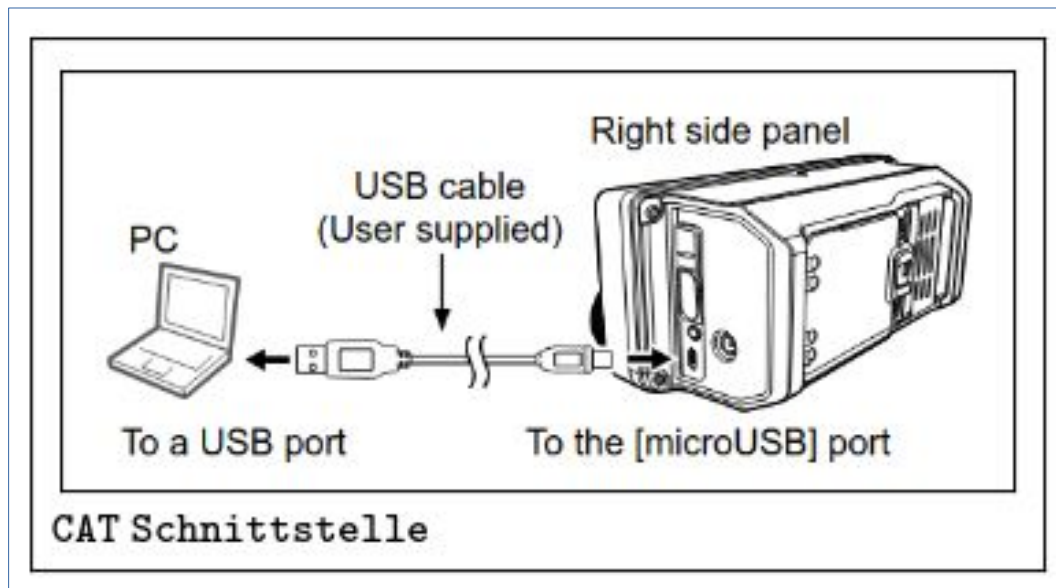
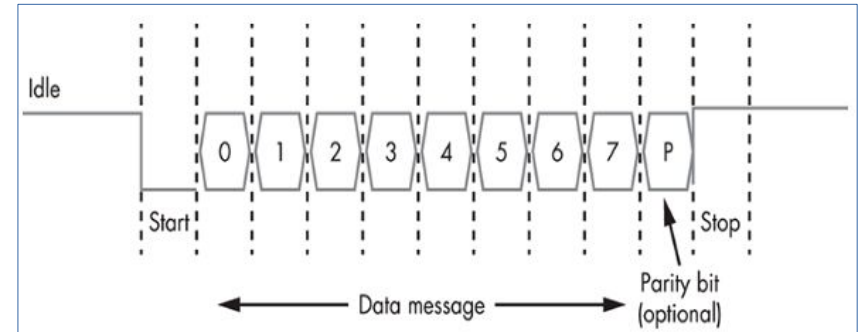
Version: 20.Jan.26

Hardware ICOM- CI-V (TTL)

CAT Steuerung = „Computer Aided Transceiver“



Datenformat „TTL-UART“

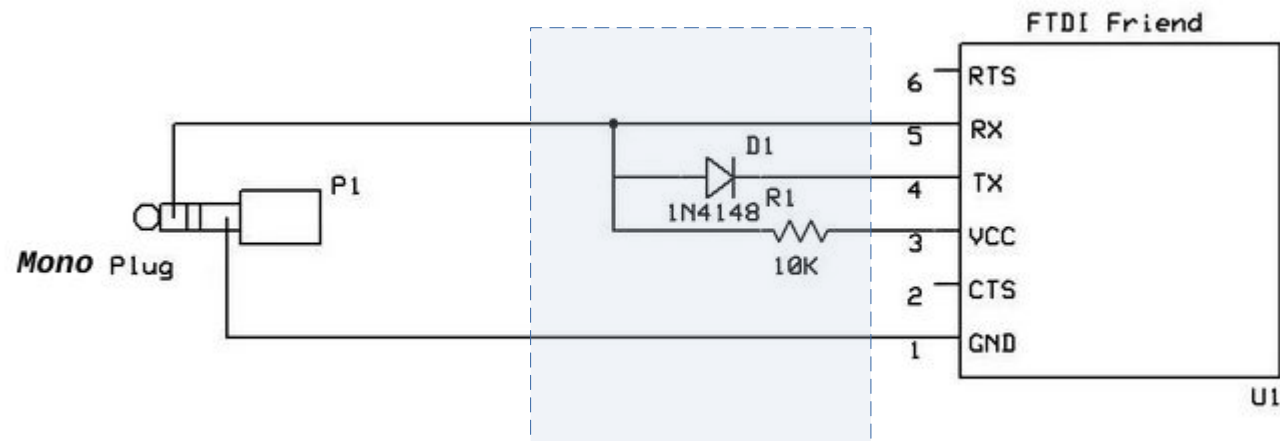


CI-V „Computer Interface V (Fünf)
TTL -Level (Ruhezustand ca +5V,
Aktivzustand ca. 0V)
Die Sende- und Empfangsleitung
(RXD und TXD) wird auf eine Leitung
kombiniert (Bus).

Beim IC-705 ist stattdessen eine
virtuelle serielle Schnittstelle
als **USB-Schnittstelle** vorhanden.

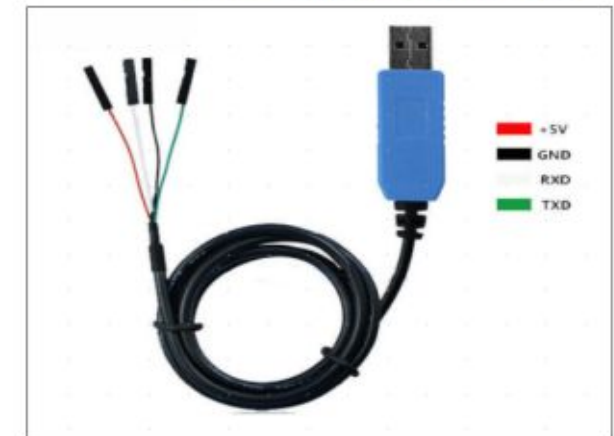
CI-V – USB-Adapter selbstgebaut

z.B. für IC-706
(bei IC-705 nicht erforderlich)



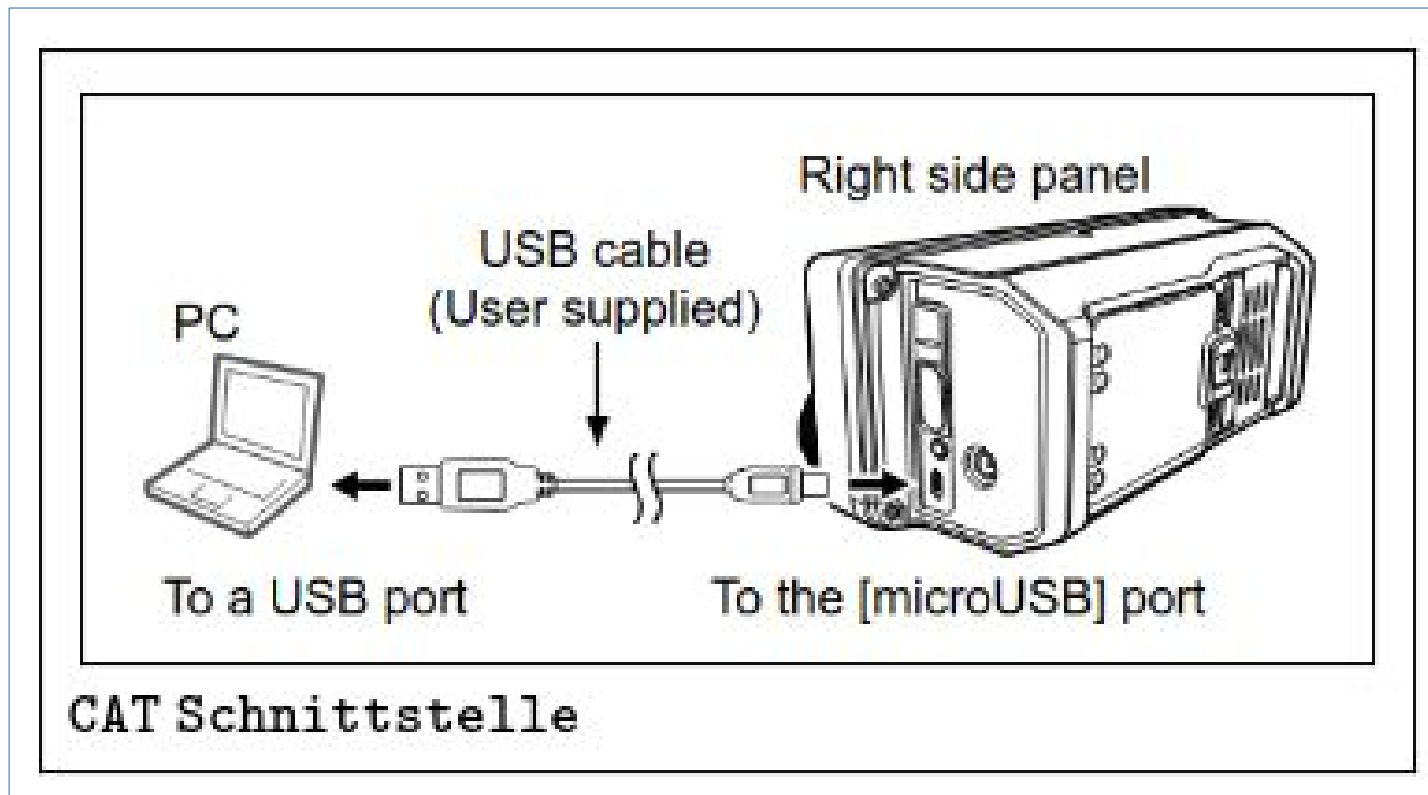
NØDIM Dec 2014

USB- Seriell-Wandler:

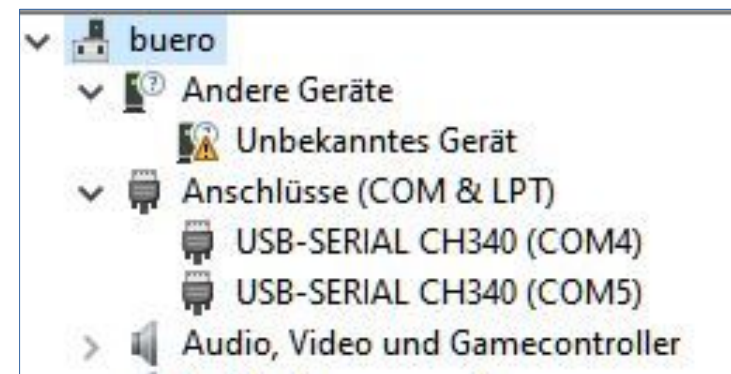


Adapter		CI-V Adapter
GND , schwarz	---	G
TXD , grün	---	rx
RXD , weiß	---	tx
+5v	---	:

USB als virtueller COM-Port



```
pi@raspi62:~/app $ ls /dev/ttyA*  
/dev/ttyACM0 /dev/ttyACM1  
pi@raspi62:~/app $ ls /dev/ttyU*  
/dev/ttyUSB0
```



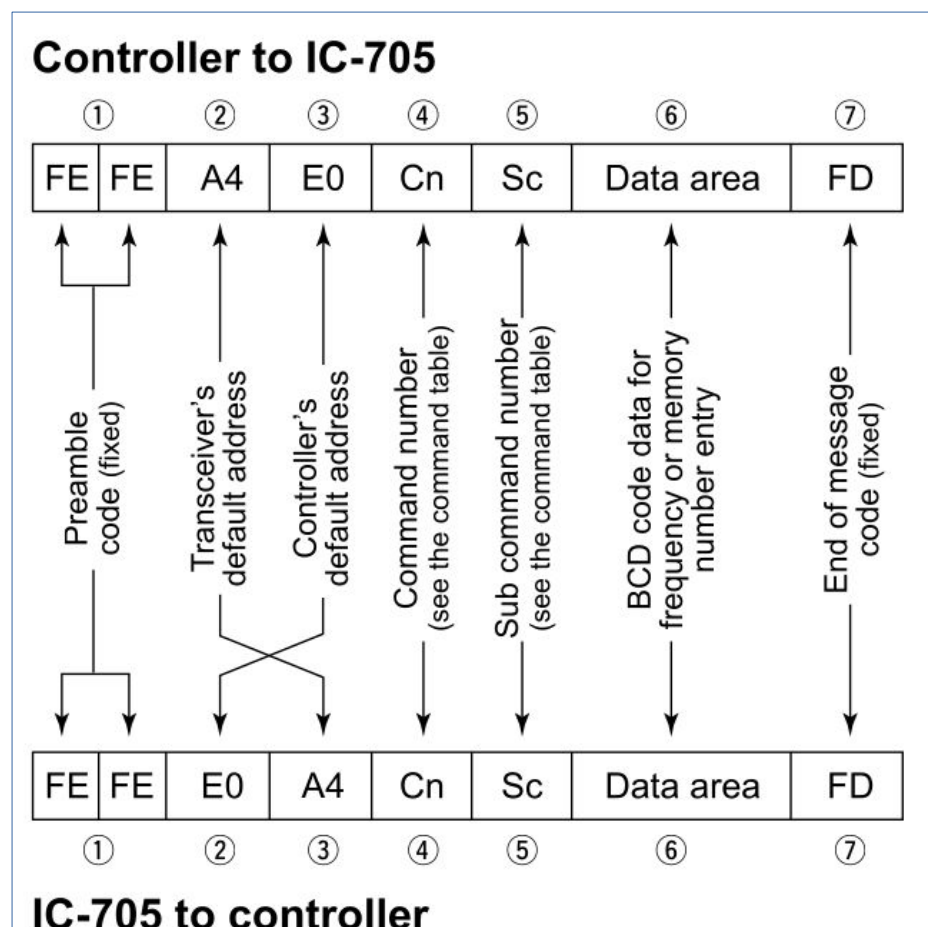
Ausschnitt IC-705-Command Table

◇ Command table

Aus:
ICOM CI-V REFERENCE GUIDE (English)

Cmd.	Sub cmd.	Data	Description
00		See p. 16.	Send the frequency data (transceive)
01		See p. 16.	Send the mode data (transceive)
02*1		See p. 16.	Read the band edge frequencies
03*1		See p. 16.	<u>Read the operating frequency</u>
04*1		See p. 16.	Read the operating mode
05*2		See p. 16.	Set the operating frequency
06*2		See p. 16.	Set the operating mode
07			Select the VFO mode

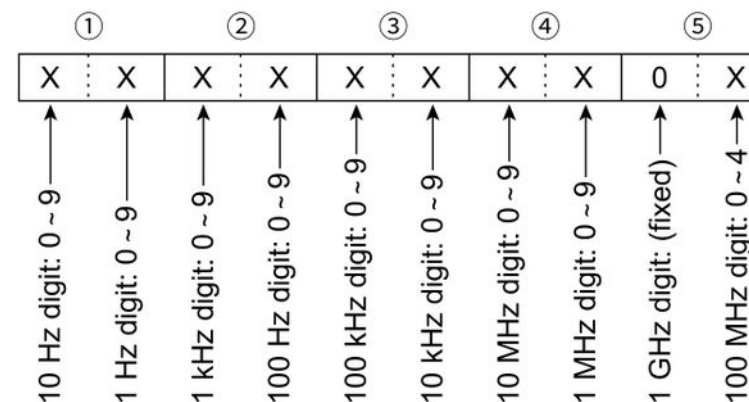
CI-V Command Format ICOM-705



◇ Command formats

• Operating frequency

Command: 00, 03, 05, 1C 03



aus:
ICOM CI-V REFERENCE GUIDE (English)

Datenformat der Frequenzabfrage im Detail

```
#ICOM Kommando zum Frequenz lesen:
# FE FE A4 E0 03 FD
# FE FE          <- kopf
# ..... A4      <- IC-705 CAT Adresse
# ..... E0      <- default ??
# ..... 03      <- Kommando Frequenz lesen
# ..... FD      <- Ende Zeichen
```

Interpretation der Antwort:

```
Antwort          = FE FE E0 A4 03 00 50 43 28 00 FD
Nutzdaten        = 00 50 43 28 00
Nutzdaten        = 00 28 43 50 00 (gedreht)
Roh-Ergebnis     = "0028435000" = text
Frequenz         = 28.435 MHz = float(text)/1e6
```

Python Skript Frequenzabfrage ICOM-705

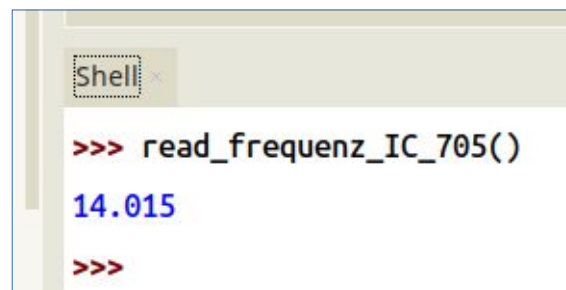
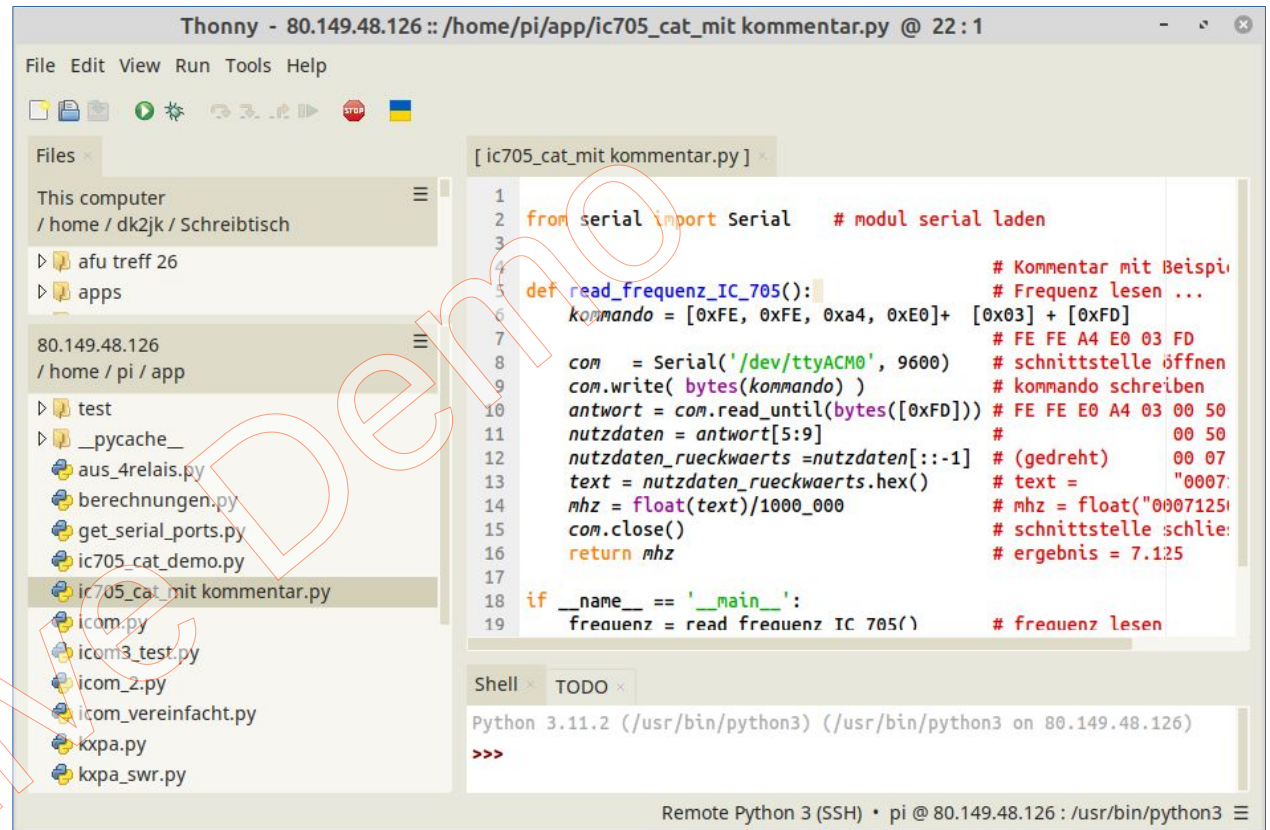
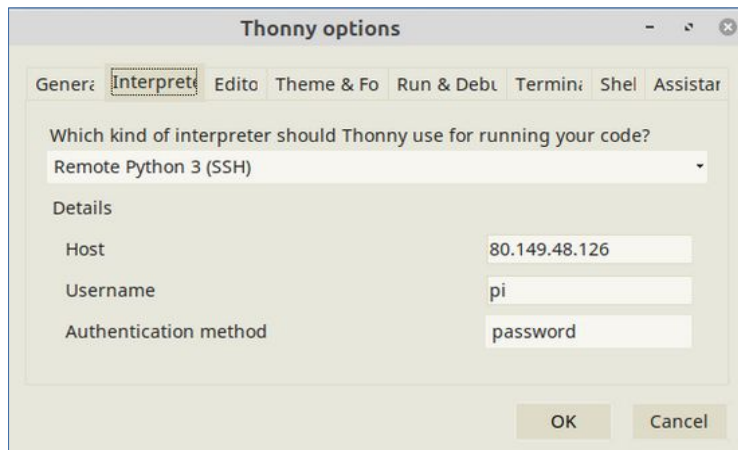
[ic705_cat_mit kommentar.py]

```
1
2 from serial import Serial    # modul serial laden
3
4                                     # Kommentar mit Beispieldaten...
5 def read_frequenz_IC_705():    # Frequenz lesen ...
6     kommando = [0xFE, 0xFE, 0xA4, 0xE0]+ [0x03] + [0xFD]
7                                     # FE FE A4 E0 03 FD
8     com = Serial('/dev/ttyACM0', 9600) # schnittstelle öffnen
9     com.write( bytes(kommando) )      # kommando schreiben
10    antwort = com.read_until(bytes([0xFD])) # FE FE E0 A4 03 00 50 12 07 00 FD
11    nutzdaten = antwort[5:9]          #          00 50 12 07 00
12    nutzdaten_rueckwaerts = nutzdaten[::-1] # (gedreht) 00 07 12 50 00
13    text = nutzdaten_rueckwaerts.hex() # text = "0007125000"
14    mhz = float(text)/1000_000        # mhz = float("0007125000")/1e6
15    com.close()                      # schnittstelle schliessen
16    return mhz                       # ergebnis = 7.125
17
18 if __name__ == '__main__':
19     frequenz = read_frequenz_IC_705() # frequenz lesen
20     print ( f' f={frequenz} MHz' )    # und anzeigen: f=7.125 MHz
```

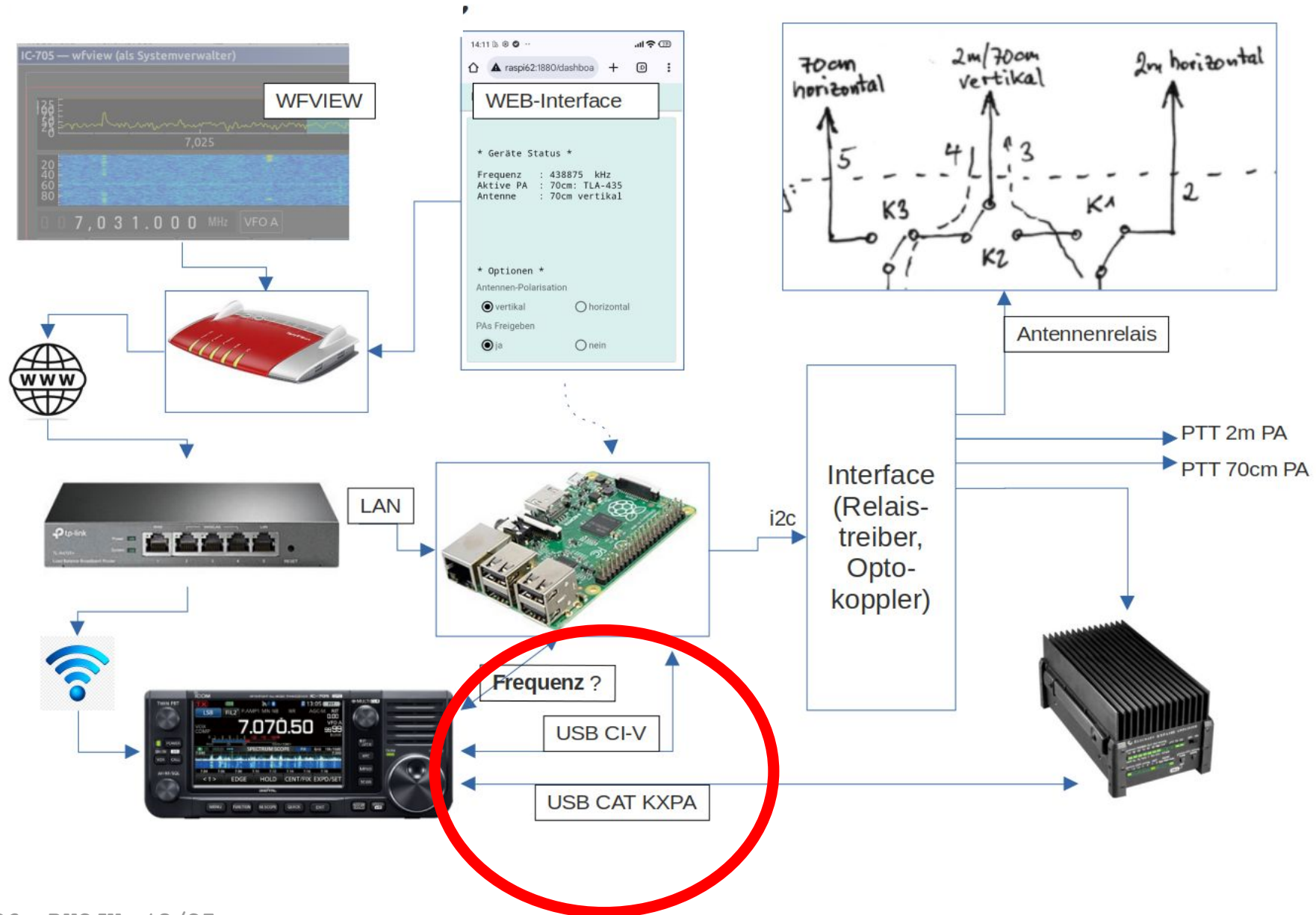
Shell

```
>>> %Run 'ic705_cat_mit kommentar.py'
f=7.125 MHz
```

Python Skript interaktiv



Wie hängt alles zusammen?



CAT-Control KXPA100

**Der Tuner wird auf die Frequenz
geschaltet noch bevor er überhaupt
HF sieht.**

Befehl: ^MT7032;

== Memory Tune 7032 kHz

Befehlsformat

KXPA100-Befehle beginnen mit einem Caret (^).
Alle Befehle verwenden den druckbaren ASCII-Zeichensatz.
Jeder serielle Befehl wird mit einem Semikolon (;)
abgeschlossen .

2. Tune-Button für ext. Tuner am FT-710

Aufgabe: Ein externer Tuner soll zum Abstimmen mit einem 10W- Träger angesteuert werden.

Möglichkeiten:

1. Manuell:

Mode=AM (ca. 25W) --> PTT am Mikrofon drücken --> Tuner Arbeitet --> zurück auf alten ,Mode'

2. Verwendung der Buchse „REM/ALC“

Signal TX_REQ auf LOW --> TX sendet Träger
Leistung reduzieren mit negativen Spannung am Signal ALC; -4.9V --> 10W. (TUN/LIN PORT SELECT auf ,Linear')

3. CAT-Schnittstelle:

eine Kommandofolge senden, die den manuellen Tune-Vorgang nachbildet.

Python Programm Frequenzabfrage YAESU

(Zum Vergleich: gleiche Aufgabe wie bei ICOM)

cat_pc_basic.py ×

```
12
13 port = '/dev/ttyUSB0'          # hier anpassen
14 baud = 38400
15
16 my_ser = serial.Serial( port, baud )
17
18 def cat( cmd ):
19     my_ser.write(cmd.encode())  # ascii in bytes umwandeln und senden
20     time.sleep(0.2)            # etwas warten
21     antwort = my_ser.read_all() # und antwort lesen
22     return antwort.decode()    # bytes in ascii umwandeln
23
```

Shell ×

```
>>> %Run cat_pc_basic.py
>>> cat('fa;')
'fa007030160;'
```

Experiment live

The image shows the Thonny IDE interface. A 'Thonny options' dialog box is open, showing the 'Interpreter' tab. The question 'Which kind of interpreter should Thonny use for running your code?' is displayed, with 'Local Python 3' selected. Below this, there is a 'Details' section with 'Python executable' listed.

The main window displays a file explorer on the left with the following files:

- This computer
 - / home / dk2jk / Schreibtisch
 - / CAT_dk2jk / pc
 - cat_pc.py
 - cat_pc_basic.py
 - get_serial_ports.py
 - icom_706_cat.py
 - Tune.desktop
 - tune_sg230.py

The code editor shows the following Python code:

```
10 '''  
11 import serial  
12 import time  
13  
14 port = '/dev/ttyUSB0' # hier anpassen  
15 baud = 38400  
16  
17 my_ser = serial.Serial( port, baud )  
18  
19 def cat( cmd = 'fa;'):  
20     my_ser.write(cmd.encode()) # ascii in bytes umwandeln und senden  
21     time.sleep(0.2) # etwas warten  
22     antwort = my_ser.read_all() # und antwort lesen  
23     return antwort.decode() # bytes in ascii umwandeln  
24
```

The shell window shows the execution of the code:

```
>>> cat()  
'fa007037800;'  
>>>
```

The status bar at the bottom right indicates 'Local Python 3 • /usr/bin/python3'.

2.1. Tune Programm am PC

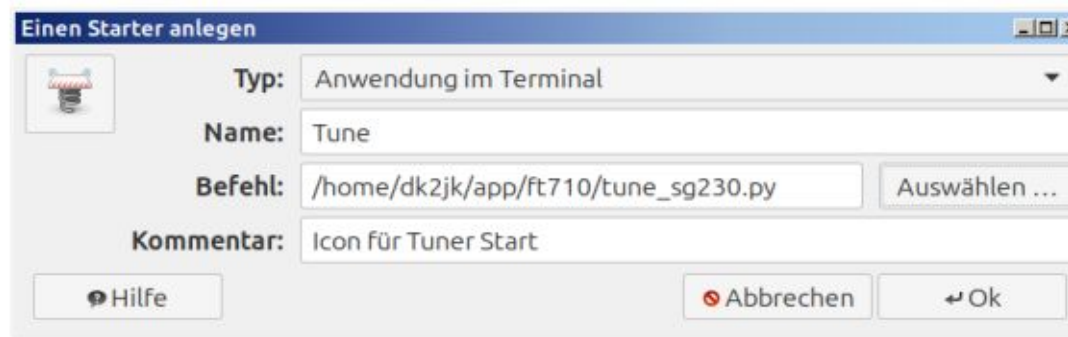
Aufgabe: eine Kommandofolge senden, die den manuellen Tune-Vorgang nachbildet

```
def tune_function():  
    meter = get('MS;') # 'Meter'- Einstellung lesen  
    mode = get('MD0;') # 'Mode' lesen  
    power = get('PC;') # 'Power' lesen  
    afgain= get('AG0;') # 'AF GAIN' lesen  
    beep = get('EX030101;') # 'beep' lesen  
    write('EX030101050;') # beep auf laut , falls aus  
    write('AG0000;') # lautstärke aus  
    write('MS50;') # 'Meter'- Einstellung auf 'SWR' stellen  
    write('PC010;') # 'Power' auf 10 Watt schalten  
    write('MD06;') # 'Mode' auf RTTY schalten  
    write('MX1;') # 'PTT' einschalten  
    sleep(tune_dauer) # x Sekunden lang Träger senden  
    write('MX0;') # 'PTT' ausschalten  
    write( mode ) # die gelesenen Werte wieder herstellen  
    write( power )  
    write( meter )  
    write( afgain )  
    write ( beep )  
    close() # Schnittstelle schliessen  
    return
```

Tune Icon (Linux)

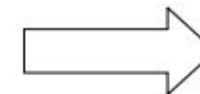
Link auf dem Linux Schreibtisch erstellen

Wählen Sie auf dem Schreibtisch mit der rechten Maustaste „Starter anlegen“ und tragen Sie unter ‚Befehl‘ den kompletten Pfad des Python-Programms ein.



Damit das Icon des „Starters“ besser aussieht, kann man im Starter-Text mit einem Texteditor folgende Zeile ergänzen: „Icon = python3“.

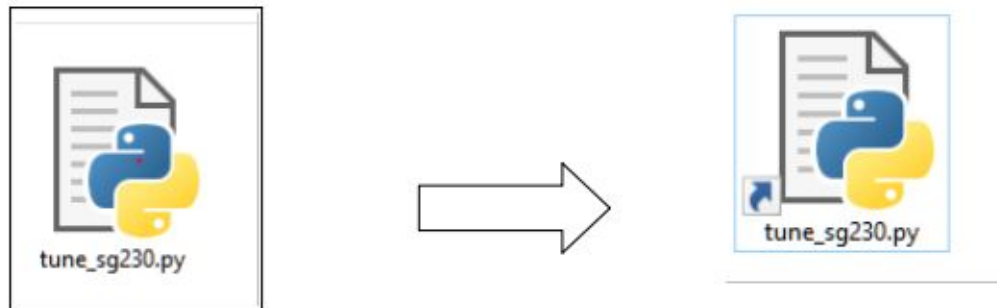
```
Tune.desktop *
1  #!/usr/bin/env xdg-open
2  [Desktop Entry]
3  Version=1.0
4  Type=Application
5  Terminal=true
6  Name[de_DE]=Tune
7  Icon = python3
8  Comment[de_DE]=Icon für Tuner Start
9  Exec=/home/dk2jk/app/ft710/tune_sg230.py
10 Name=Tune
11 Comment=Icon für Tuner Start
12
```



Tune Icon Link (Windows)

Link auf dem Windows Desktop erstellen

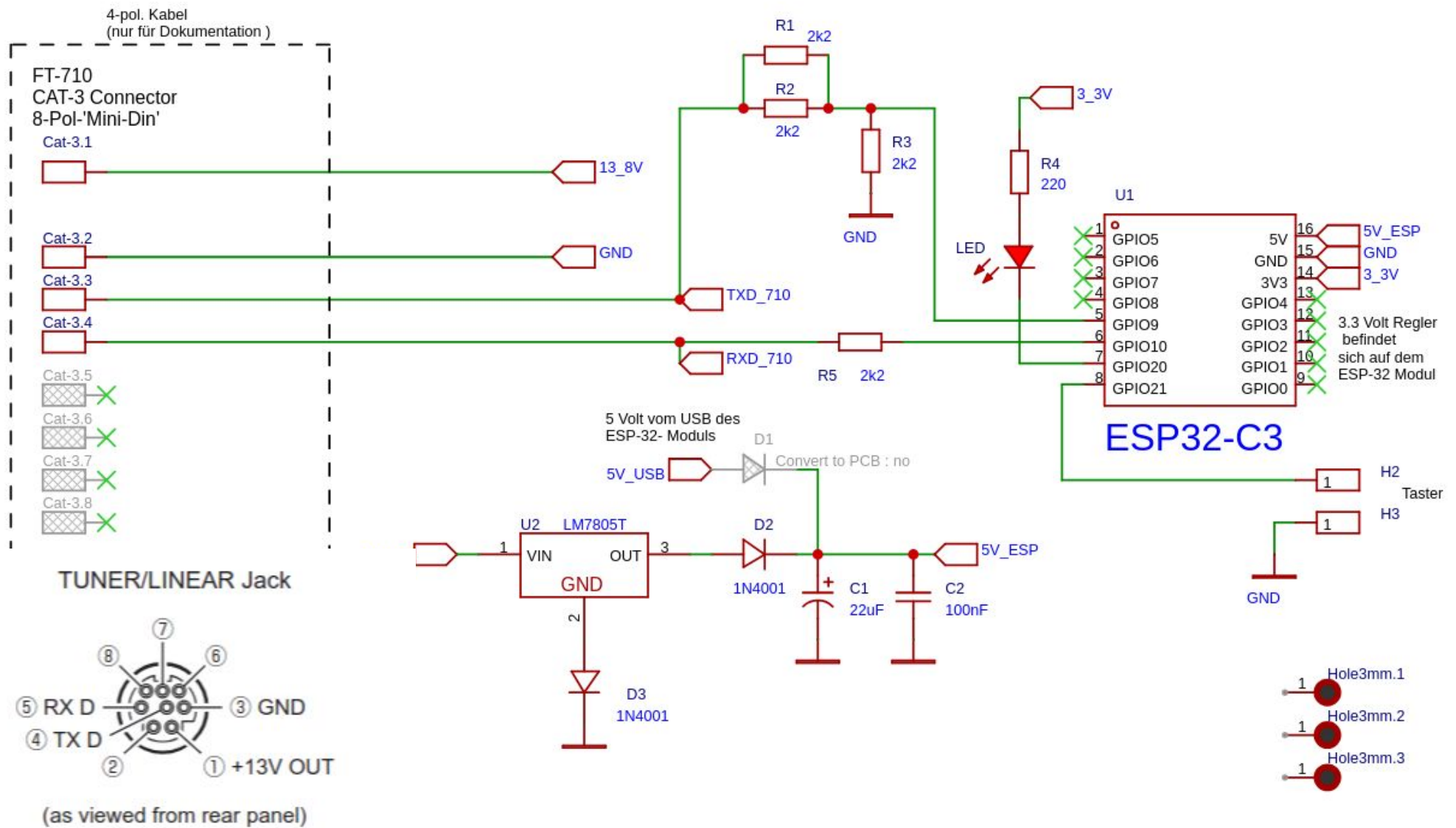
Wähle im Dateimanager das Programm aus und erstelle mit der rechten Maustaste eine Verknüpfung:



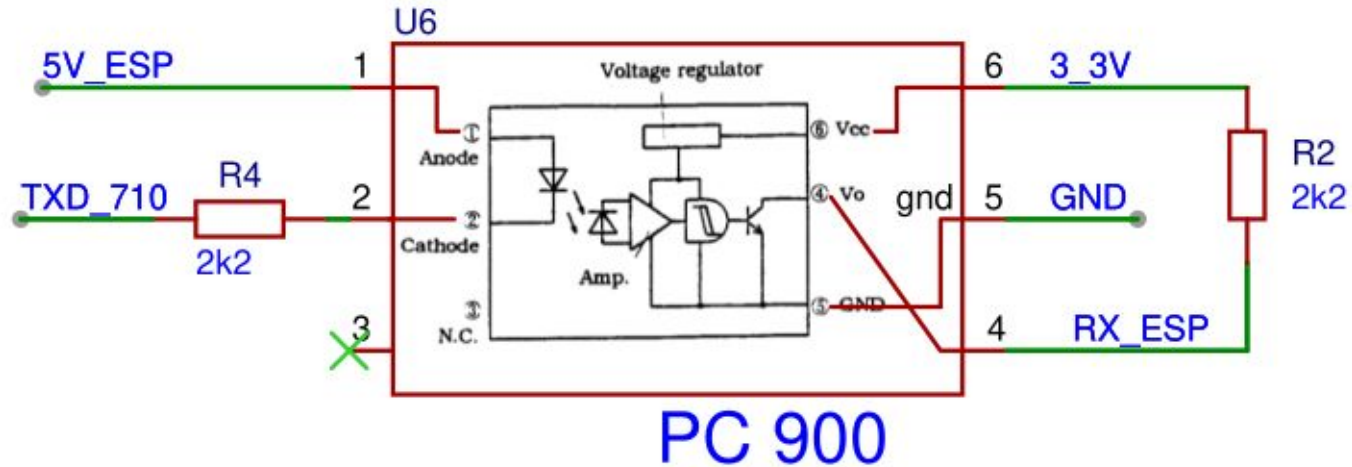
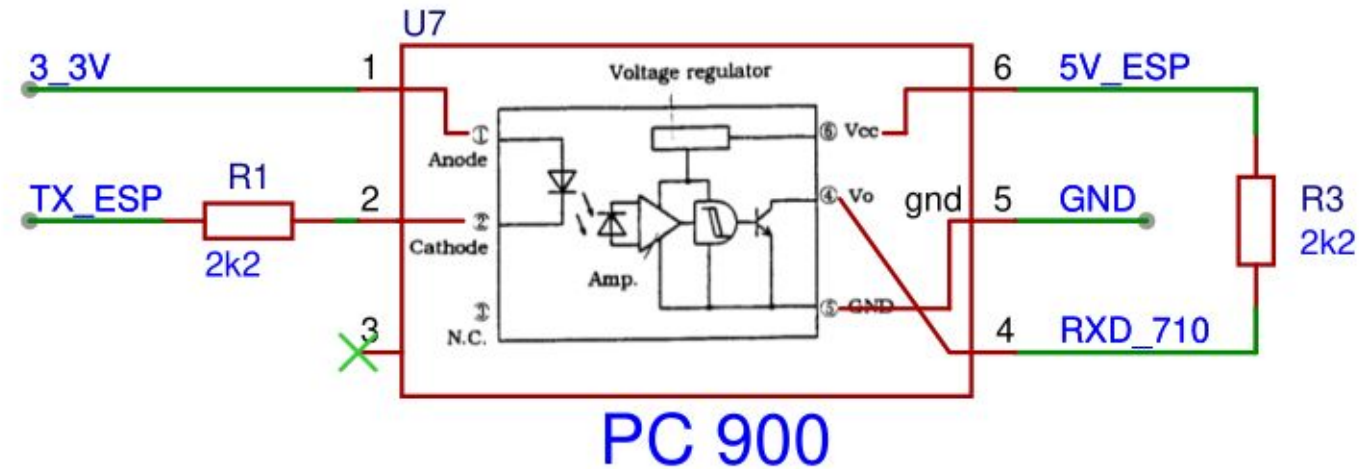
Die Verknüpfung wird als „Tuner-Start-Button“ auf den Desktop gezogen.

Beim Start über den Desktop öffnet sich ein Terminal- Fenster mit Statusanzeigen..

2.2. Tune Taste mit ESP-32



Variante mit Optokoppler



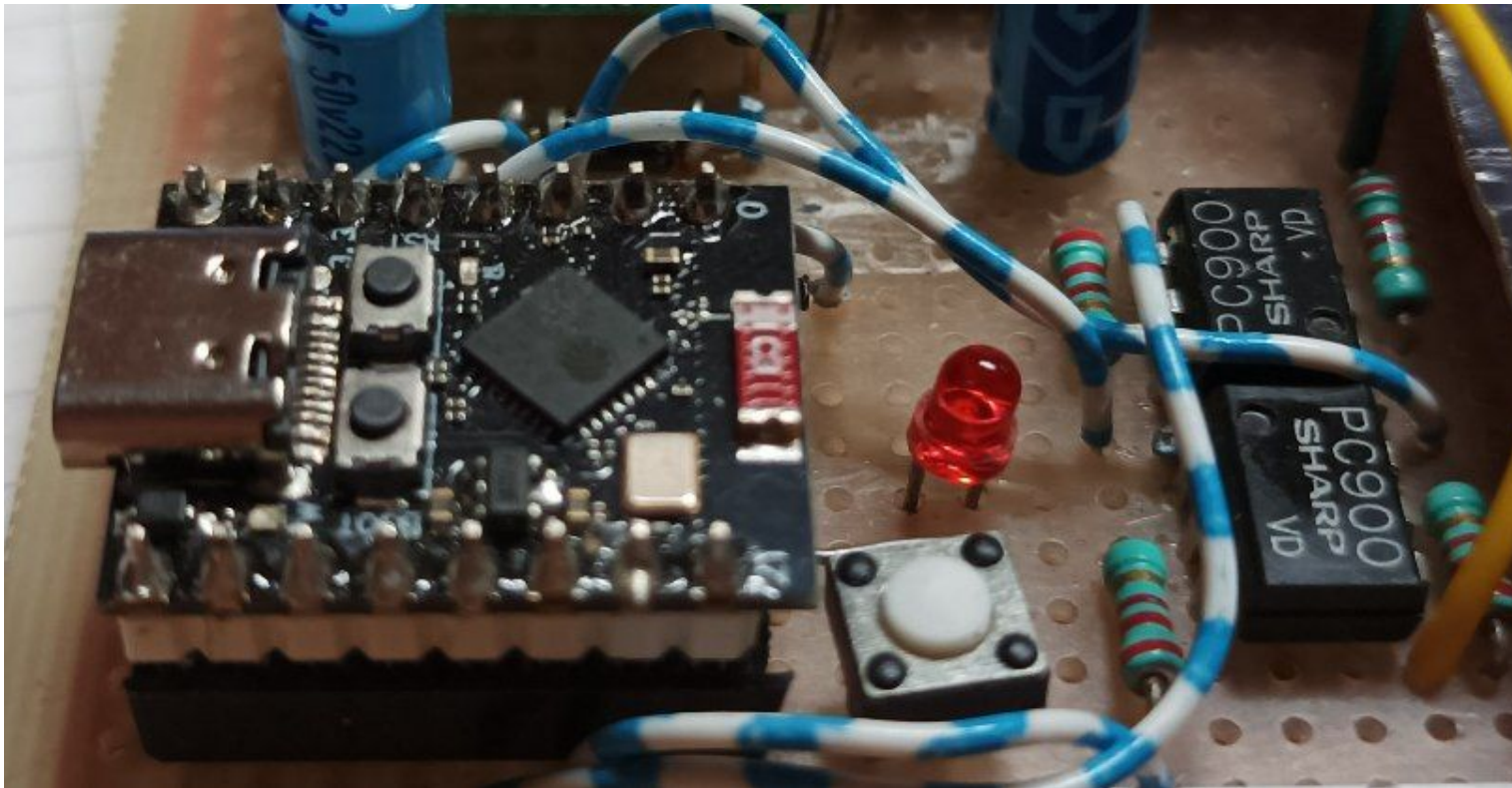
MicroPython Programm ESP32

```
def tune_function():  
    meter = get('MS;') # 'Meter'- Einstellung lesen  
    mode = get('MD0;') # 'Mode' lesen  
    power = get('PC;') # 'Power' lesen  
    afgain= get('AG0;') # 'AF GAIN' lesen  
    beep = get('EX030101;') # 'beep' lesen  
    write('EX030101050;') # beep auf laut , falls aus  
    write('AG0000;') # lautstärke aus  
    write('MS50;') # 'Meter'- Einstellung auf 'SWR' stellen  
    write('PC010;') # 'Power' auf 10 Watt schalten  
    write('MD06;') # 'Mode' auf RTTY schalten  
    write('MX1;') # 'PTT' einschalten  
    sleep(tune_dauer) # x Sekunden lang Träger senden  
    write('MX0;') # 'PTT' ausschalten  
    write( mode ) # die gelesenen Werte wieder herstellen  
    write( power )  
    write( meter )  
    write( afgain )  
    write( beep )  
    close()  
    return
```

esp32_tune.py

```
while True:  
    x= taste() # taste lesen , low aktiv  
    led(x) # led an solange taste gedrueckt ist  
    if x ==0: # taste gedrueckt ?  
        tune_function()
```

Bild „Tune Button mit ESP32“



Verweise

Alle Unterlagen auf Github:

<https://github.com/dk2jk/CAT--Control>

Python Download für verschiedene Betriebssysteme:

<https://www.python.org/downloads/>

Thonny Python- Entwicklungs- Oberfläche (IDE):

<https://thonny.org/>

MicroPython Download für Microcontroller:

<https://micropython.org/download/>

ICOM CI-V REFERENCE GUIDE (English):

<https://www.icomjapan.com/support/manual/3064/>

YAESU CAT Reference:

https://www.radiomanual.info/schemi/YAESU_HF/FT-710_CAT_user.pdf

Elecraft KXPA100 Amplifier Command Reference:

<https://ftp.elecraft.com/KXPA>